

Assignment 1: Monty Hall Simulation

General

You complete this assignment alone or in pairs. If you work in pairs, each person must submit the assignment. At the top of your work, include a comment line stating with whom you collaborated.

This assignment counts for **12/30** of the daily work for the module: Learning to Program in Python, functional programming assignment.

You will be assessed on the correct functioning of the code, efficiency, and readability.

For this assignment you may use all course material provided in this module. You may also conduct online research to solve sub-problems. For example: how do I pick a random element from a list? How do I iterate alphabetically over a dictionary? etc. Do not look up complete solutions to the exercises. You may not use AI (ChatGPT, Copilot, ...) to generate code for (parts of) the solutions.

Submit a Python file (.py) or a Jupyter Notebook file (.ipynb) no later than **2 November** via Syntra Cloud.

Context

The Monty Hall problem is a brain teaser based on an American television game show from the 1960s hosted by Monty Hall. It proceeds as follows. There are three doors, with a prize behind one door and nothing behind the other two. Importantly, the host knows which door hides the prize. The contestant chooses one of the three doors; it is not opened yet. The host then opens one of the two remaining doors that does not contain the prize. Two unopened doors remain. The contestant is asked whether they want to keep their original choice or switch to the other door. The question is: is it better to stick with the original choice, to switch, or does it make no difference?

Assignment

Write a program that simulates the Monty Hall problem.

Your program must run the simulation n times where you stay with your initial door, and n times where you switch doors (with n a value the user can provide). As output, display the percentage of wins for each of the two strategies.

Not part of the assignment: Now try your program with $n = 4$, $n = 400$, $n = 1000$. The more runs you perform, the closer you will approximate the true probability distribution. Are the results what you expected? Can you explain them? Don't worry if you can't right away — there are Nobel Prize winners who were fooled by this problem.